

Hub de Estudos

Apostilas, exercícios e simulados

</> HTML

{ ; } CSS

JS JavaScript

Git

Desenvolvimento Web Front-End

HTML, CSS, JavaScript e TypeScript



Desenvolvimento Web Front-End

HTML, CSS, JavaScript e TypeScript

Hub de Estudos

Apostilas, exercícios e simulados

2026

Todos os direitos reservados. Nenhuma parte desta publicação pode ser reproduzida, armazenada ou transmitida de qualquer forma, ou por qualquer meio, sem autorização prévia.

Desenvolvido por TecPro Tecnologias.

Sumário

Capítulo 1 – Como a Web funciona **4**
1.1 Internet e Web 4
1.2 Cliente e servidor 4
1.3 O endereço: a URL 5
1.4 Do nome ao número: o DNS 6
1.5 O HTTP: pedido e resposta 6
1.6 Front-end e back-end 7
1.7 Como o navegador monta a página 8
1.8 As ferramentas de trabalho 8
1.9 A primeira página 9
Erros Comuns 11

Capítulo 1 – Como a Web funciona

Antes de escrever a primeira linha de HTML, vale entender o caminho que uma página percorre até aparecer na tela: quem a guarda, como ela é pedida, como chega e o que o navegador faz com ela. Neste capítulo você vai conhecer a diferença entre a Internet e a Web, o modelo cliente-servidor, as partes de um endereço, o DNS, o protocolo HTTP e as etapas que o navegador percorre para desenhar uma página. No final, você vai preparar as ferramentas de trabalho e criar a primeira página, com um arquivo de HTML, um de CSS e um de JavaScript.

1.1 Internet e Web

As duas palavras costumam ser usadas como sinônimos, mas não são a mesma coisa. A **Internet** é a rede: milhões de computadores do mundo inteiro ligados por cabos, fibras e antenas, que trocam dados seguindo um conjunto de regras comum, os protocolos TCP/IP. Ela existe desde os anos 1970 e carrega muitos serviços diferentes: e-mail, mensagens, chamadas de vídeo, jogos.

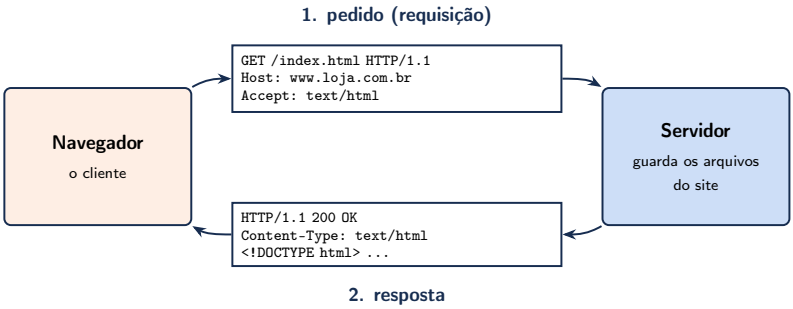
A **Web** (*World Wide Web*) é um desses serviços: um conjunto de documentos, as **páginas**, que ficam guardados em computadores espalhados pela Internet e são ligados uns aos outros por **links**. Ela foi proposta em 1989 por Tim Berners-Lee, um pesquisador do CERN, o laboratório europeu de física de partículas, para que os cientistas compartilhassem documentos. Em 1991, ele publicou o primeiro site, junto com as três peças que sustentam a Web até hoje:

- o **HTML**, a linguagem em que as páginas são escritas;
- a **URL**, o endereço único de cada página;
- o **HTTP**, o protocolo que o navegador usa para pedir uma página e o servidor usa para entregá-la.

As regras dessas tecnologias são definidas por organizações abertas. O **WHATWG**, formado pelas empresas que fazem os navegadores, mantém o HTML como um **padrão vivo** (*living standard*): não existe mais uma versão "final", e o padrão é atualizado continuamente. O nome HTML5, ainda muito usado, se refere à grande atualização publicada em 2014 e virou sinônimo de "HTML moderno". O **W3C** mantém o CSS e outros padrões. Para consultar como cada recurso funciona, a referência mais usada é o **MDN Web Docs** (`developer.mozilla.org`), mantido pela Mozilla, com tradução para o português em boa parte das páginas.

1.2 Cliente e servidor

Toda visita a um site envolve dois papéis. O **cliente** é quem pede: no caso da Web, o **navegador**, como o Chrome, o Firefox, o Edge ou o Safari. O **servidor** é um computador, ligado o tempo todo à Internet, que guarda os arquivos do site e os entrega a quem pedir. A conversa entre os dois é sempre a mesma: o cliente faz um **pedido** (a **requisição**) e o servidor devolve uma **resposta**.



depois, um novo pedido para cada arquivo citado no HTML: CSS, JavaScript, imagens

Uma página quase nunca é um arquivo só. O HTML cita outros arquivos, como folhas de estilo, *scripts* e imagens, e o navegador faz um novo pedido para cada um deles, um atrás do outro ou vários ao mesmo tempo. Uma página de um grande portal chega a fazer centenas de pedidos até ficar pronta.

O servidor pode entregar dois tipos de conteúdo. Um site **estático** é formado por arquivos prontos, iguais para todas as pessoas: o servidor só os encontra e envia. Um site **dinâmico** tem um programa no servidor que monta a resposta na hora, de acordo com quem pede, consultando um banco de dados: é assim que uma loja mostra o seu carrinho e uma rede social mostra o seu *feed*. Nesta apostila, você vai construir a parte que roda no navegador; a parte que roda no servidor é assunto das disciplinas de Web II e Web III.

1.3 O endereço: a URL

Cada recurso da Web tem um endereço, a **URL** (*Uniform Resource Locator*). Uma URL completa tem várias partes:

```
https://www.loja.com.br:443/produtos?busca=caneta#lista
\___/   \______________/ \_/\_______/\___________/\____/
  |            |         |     |          |         |
protocolo   domínio   porta caminho   consulta  fragmento
```

Parte	O que indica
protocolo	as regras da conversa: `https`, ou `http`, sem criptografia
domínio	o nome do servidor, como `www.loja.com.br`
porta	a "porta de entrada" do servidor; quase sempre omitida, porque o `https` usa a 443 e o `http`, a 80
caminho	qual recurso, dentro do servidor, como `/produtos`
consulta	informações extras, em pares `nome=valor`, depois do `?` e separadas por `&`
fragmento	um ponto dentro da página, depois do `#`; ele nem é enviado ao servidor, e só o navegador o usa, para rolar a página até aquele ponto

O próprio JavaScript sabe separar essas partes. Este trecho foi executado no Node.js, que roda JavaScript fora do navegador, com o resultado em comentário:

```
const url = new URL(
  "https://www.loja.com.br:443/produtos?busca=caneta#lista",
);
url.protocol;  // "https:"
url.hostname;  // "www.loja.com.br"
url.port;      // ""  (443 é a porta padrão do https)
url.pathname;  // "/produtos"
url.search;    // "?busca=caneta"
url.hash;      // "#lista"
url.searchParams.get("busca");  // "caneta"
```

Repare na porta vazia: como 443 é a porta padrão do `https`, ela nem precisa aparecer, e o navegador a remove. Os endereços que você vai usar no seu computador, como `http://localhost:3000`, são diferentes: `localhost` é o nome do próprio computador, e a porta precisa aparecer, porque não é a padrão.

1.4 Do nome ao número: o DNS

Na Internet, os computadores se encontram por números, os **endereços IP**, como `104.20.23.154`. Nomes como `example.com` existem para as pessoas. Quem traduz um no outro é o **DNS** (*Domain Name System*), uma espécie de lista telefônica distribuída por servidores do mundo inteiro. O comando `nslookup`, presente no Windows, no Linux e no macOS, faz essa consulta:

```
$ nslookup example.com
...
Nome:    example.com
Addresses:  2606:4700:10::ac42:93f3
            2606:4700:10::6814:179a
            104.20.23.154
            172.66.147.243
```

Um mesmo nome pode ter vários endereços: grandes sites espalham cópias por vários servidores, e o DNS distribui as pessoas entre eles. Os endereços com dois-pontos estão no formato mais novo, o IPv6; os outros, no formato tradicional, o IPv4. O resultado de cada consulta fica guardado por um tempo no seu computador, para que a próxima visita não precise perguntar de novo.

1.5 O HTTP: pedido e resposta

O **HTTP** (*HyperText Transfer Protocol*) define o formato das mensagens trocadas entre o navegador e o servidor. Elas são texto, e dá para vê-las com o `curl`, um programa de linha de comando que faz pedidos HTTP, presente no Windows, no Linux e no macOS. A opção `-I` pede só o cabeçalho da resposta:

```
$ curl -I https://example.com
HTTP/1.1 200 OK
```

```
Date: Sun, 18 Oct 2026 02:01:44 GMT
Content-Type: text/html; charset=utf-8
Server: cloudflare
last-modified: Tue, 13 Oct 2026 20:08:28 GMT
```

A primeira linha traz o **código de status**, um número de três dígitos que resume o resultado, e as seguintes são os **cabeçalhos**, informações sobre a resposta no formato `Nome: valor`. O `Content-Type` é o mais importante para o navegador: `text/html` diz que o conteúdo é uma página; para uma folha de estilo, seria `text/css`, e para uma imagem, `image/png`, por exemplo. Depois dos cabeçalhos vem o **corpo**, com o conteúdo em si, que o `-I` omitiu.

O pedido diz **o que** se quer fazer com um **método**. Os dois que você vai encontrar nesta apostila são o `GET`, para buscar algo, usado em toda página aberta e em todo link clicado, e o `POST`, para enviar dados, usado nos formulários (Capítulo 3). Os códigos de status são agrupados pelo primeiro dígito:

Faixa	Significado	Exemplos comuns
`2xx`	deu certo	`200 OK`
`3xx`	o recurso está em outro lugar	`301 Moved Permanently`, `304 Not Modified`
`4xx`	erro de quem pediu	`404 Not Found`, `403 Forbidden`
`5xx`	erro do servidor	`500 Internal Server Error`

O **HTTPS** é o HTTP protegido por criptografia. Sem ele, qualquer pessoa no caminho, como alguém na mesma rede Wi-Fi de uma lanchonete, pode ler e até alterar o que trafega, inclusive senhas. Com ele, só o navegador e o servidor entendem as mensagens. Hoje praticamente todos os sites usam HTTPS, e os navegadores marcam como "não seguros" os que não usam. Os detalhes do HTTP, como os outros métodos e o uso dos códigos de status em APIs, são estudados a fundo na disciplina de Web III.

1.6 Front-end e back-end

O trabalho de construir um site se divide em duas frentes. O **back-end** é o que roda no servidor: o programa que monta as páginas dinâmicas, guarda os dados em um banco, confere senhas e aplica as regras do negócio. O **front-end** é o que roda no navegador: tudo o que a pessoa vê e com que interage. Esta apostila é sobre o front-end, que se apoia em três linguagens, cada uma com um papel:

Linguagem	Papel	Em uma casa, seria...
HTML	a estrutura e o significado do conteúdo	as paredes, as portas e os cômodos
CSS	a aparência: cores, fontes, espaços e posições	a pintura, os móveis e a decoração
JavaScript	o comportamento: reagir a cliques, buscar dados, mudar a página	a parte elétrica: o interruptor que acende a luz

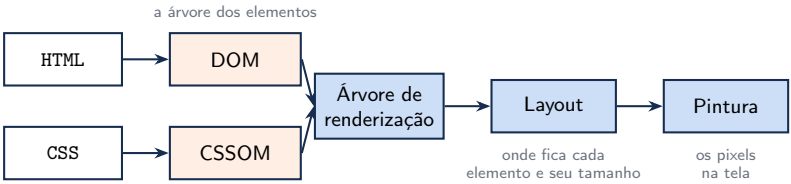
O HTML é uma **linguagem de marcação**: ele descreve o que cada parte do conteúdo é, um título, um parágrafo, uma lista, mas não faz cálculos nem toma decisões. O CSS é uma **linguagem de estilo**. Só o JavaScript é uma **linguagem de programação**,

com variáveis, condições e repetições. Manter cada uma no seu papel, em arquivos separados, deixa o site mais fácil de manter: para mudar a cor de todos os títulos, basta alterar uma linha do CSS, sem mexer no conteúdo.

Mesmo as ferramentas mais modernas, como o React e o Next.js, que você vai estudar na disciplina de Web II, geram HTML, CSS e JavaScript no fim das contas, porque é isso que os navegadores entendem. Quem domina as três linguagens entende o que essas ferramentas produzem e consegue resolver os problemas que elas não resolvem sozinhas.

1.7 Como o navegador monta a página

Entre receber o HTML e mostrar a página, o navegador percorre algumas etapas, chamadas em conjunto de **caminho crítico de renderização**:



1. O navegador lê o HTML e monta o **DOM** (*Document Object Model*), uma árvore em que cada elemento da página é um nó, dentro do seu elemento "pai". É no DOM que o JavaScript mexe para mudar a página (Capítulo 9).
2. Ao encontrar uma folha de estilo, ele a baixa e monta o **CSSOM**, a árvore das regras de estilo.
3. Juntando as duas, ele monta a **árvore de renderização**, só com o que vai aparecer, e já com o estilo de cada elemento.
4. No **layout**, ele calcula a posição e o tamanho de cada elemento, de acordo com a largura da tela.
5. Na **pintura**, ele desenha os pixels: textos, cores, bordas e imagens.

Esse caminho explica algumas práticas que você vai ver ao longo da apostila. Um *script* comum interrompe a leitura do HTML até ser baixado e executado; com o atributo `defer`, ele é baixado em paralelo e só roda quando o HTML terminou de ser lido. E sempre que o JavaScript muda a página, o navegador refaz as etapas de layout e pintura das partes afetadas, o que tem um custo quando é feito muitas vezes seguidas.

1.8 As ferramentas de trabalho

Para desenvolver para a Web, bastam três ferramentas, todas gratuitas:

- um **editor de código**. O mais usado é o **Visual Studio Code** (VS Code), da Microsoft, para Windows, Linux e macOS. Ele colore o código, completa as *tags* enquanto você digita e mostra erros;
- um **navegador moderno**, como o Chrome, o Edge ou o Firefox, com as ferramentas de desenvolvedor;
- um **servidor local**, para abrir as páginas como elas seriam abertas na Web.

As **ferramentas de desenvolvedor** (*DevTools*) abrem com a tecla `F12`, ou com o botão direito do mouse sobre a página, na opção **Inspecionar**. Três abas vão ser usadas o tempo todo:

Aba	Para que serve
Elements	mostra o DOM e o CSS de cada elemento; dá para alterar os dois ali mesmo e ver o efeito na hora, sem mexer nos arquivos
Console	mostra os erros do JavaScript e permite executar comandos
Network	lista cada pedido feito pela página, com o método, o status, o tipo e o tempo

Uma página pode ser aberta com um duplo clique no arquivo, e o endereço passa a começar com `file://`. Para as primeiras páginas, isso funciona, mas alguns recursos, como os pedidos do Capítulo 10, só funcionam quando a página é servida por HTTP. O jeito mais simples de ter um servidor no seu computador é a extensão **Live Server** do VS Code: com ela instalada, o botão **Go Live**, na barra inferior, abre a página no navegador e a recarrega a cada arquivo salvo. Quem já tem o Node.js instalado tem outra opção, sem extensão nenhuma, digitando no terminal, dentro da pasta do site:

```
$ npx serve
 INFO  Accepting connections at http://localhost:3000
```

1.9 A primeira página

Crie uma pasta chamada `primeira-pagina` e, dentro dela, três arquivos. O primeiro é o HTML, com o conteúdo:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>Minha primeira página</title>
  <link rel="stylesheet" href="estilo.css">
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Olá, Web!</h1>
  <p>Esta página tem três arquivos: HTML, CSS e JavaScript.</p>
  <button type="button" id="botao">Clique aqui</button>
  <p id="mensagem"></p>
</body>
</html>
```

As linhas antes do `<body>` são o "cabeçalho" do documento, com informações para o navegador: o idioma, a codificação dos caracteres, o título que aparece na aba e os arquivos de estilo e de *script* usados. Cada uma delas é explicada no Capítulo

2. O conteúdo visível fica dentro do `<body>`: um título, um parágrafo, um botão e um parágrafo vazio, que vai receber uma mensagem. O segundo arquivo, `estilo.css`, define a aparência:

```
body {
  font-family: system-ui, sans-serif;
  max-width: 40rem;
  margin: 2rem auto;
  padding: 0 1rem;
}

h1 {
  color: #24124f;
}

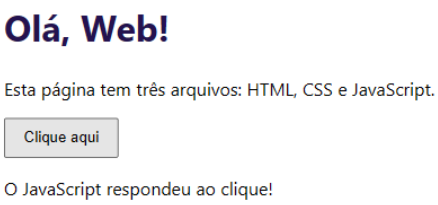
button {
  padding: 0.5rem 1rem;
  cursor: pointer;
}
```

E o terceiro, `script.js`, dá vida ao botão:

```
const botao = document.querySelector("#botao");
const mensagem = document.querySelector("#mensagem");

botao.addEventListener("click", () => {
  mensagem.textContent = "O JavaScript respondeu ao clique!";
});
```

O JavaScript procura o botão e o parágrafo vazio pelos seus `id`, e, a cada clique no botão, escreve uma frase no parágrafo. Abra a página com o Live Server, ou com o `npx serve`, e clique no botão:



Na aba Network das ferramentas de desenvolvedor, recarregue a página. A lista mostra os três pedidos que o navegador fez, um para cada arquivo, cada um com o seu tipo:

```
GET /           200  text/html
GET /estilo.css 200  text/css
GET /script.js  200  text/javascript
```

O primeiro pedido, para `/`, devolveu o `index.html`: quando o caminho é uma pasta, os servidores procuram, por convenção, um arquivo com esse nome dentro dela. Os outros dois foram feitos porque o HTML os citou. Talvez apareça também um quarto pedido, para `/favicon.ico`, com status `404`, e um erro no Console: o navegador procura sozinho o ícone que aparece na aba, e a página ainda não tem um. Esse erro é inofensivo, e o Capítulo 2 mostra como declarar o ícone. Se você errar o nome de um arquivo, como `estilos.css` em vez de `estilo.css`, a página aparece sem estilo, e a aba Network mostra o pedido com o status `404`: é o primeiro lugar a olhar quando algo não carrega.

Por fim, uma página pode estar "funcionando" na tela e, mesmo assim, ter erros que um navegador perdoa e outro não. O **validador do W3C** (`validator.w3.org`) confere o HTML contra o padrão. Esta página foi conferida com ele e com o `html-validate`, outra ferramenta de validação, que acusou um detalhe na primeira versão: o `<button>` estava sem o atributo `type`. Sem ele, um botão dentro de um formulário envia o formulário ao ser clicado, o que nem sempre é o desejado; por isso a página declara `type="button"`, assunto do Capítulo 3.

Erros Comuns

- **Confundir a Internet com a Web.** A Internet é a rede; a Web é um dos serviços que funcionam sobre ela.
- **Achar que o HTML é uma linguagem de programação.** Ele descreve o conteúdo; quem programa o comportamento é o JavaScript.
- **Errar o nome ou a pasta de um arquivo citado no HTML.** O navegador recebe um `404`, e a página aparece sem estilo ou sem *script*. Confira a aba Network.
- **Misturar as três linguagens em um arquivo só.** Estilo e *script* espalhados no HTML tornam qualquer mudança trabalhosa. Separe cada um no seu arquivo.
- **Esquecer o `defer` no *script* do `<head>`.** O *script* roda antes de o HTML existir, e o `querySelector` não encontra o elemento.
- **Testar só abrindo o arquivo com duplo clique.** Alguns recursos só funcionam com a página servida por HTTP. Use um servidor local.
- **Ignorar o Console.** Um erro de JavaScript costuma estar escrito lá, com o arquivo e a linha.